

TASKORA

Team Management Application

SYSTEM DESIGN DOCUMENT

Prepared by:

Raynald (Programmer) | Duard (System Analyst)
Yovela (Product Owner) | Nicholas (Tech Lead)

1. Introduction

1.1 Purpose

This System Design Document (SDD) defines the technical architecture, UML diagrams, and database schema for Taskora, a centralized team management application designed to help university students organize group projects, assign tasks, track contributions, and communicate within a single platform.

1.2 Technology Stack

Component	Technology	Rationale
Frontend	ReactJS + Vite	Component-based UI with fast HMR and optimized builds
Backend	TypeScript + Express.js + Socket.io	Typed API layer with real-time bidirectional communication
Database	PostgreSQL	Relational integrity for structured project/task data
Deployment	Vercel	Production hosting with CI/CD integration
Architecture	Layered Architecture	Routes → Controllers → Services → Repositories → Models

2. System Overview

2.1 Architecture

Taskora follows a layered architecture that separates concerns horizontally across five layers. The React frontend communicates with the Express.js backend via REST APIs and a persistent Socket.io connection for real-time events. The backend enforces role-based access at the controller layer and delegates business logic to the service layer, which interacts with PostgreSQL through repository classes.

Layer	Responsibility
Routes	HTTP endpoint definitions and Socket.io event bindings. Maps incoming requests to the correct controller.
Controllers	Request validation, authentication checks, and role-based authorization. Calls services and returns HTTP responses.
Services	Core business logic — task state transitions, contribution calculation, notification triggers, activity logging.
Repositories	Data access layer. Constructs and executes parameterized SQL queries against PostgreSQL.
Models	TypeScript interfaces and type definitions for all entities (User, Task, Workspace, etc.).

2.2 User Roles

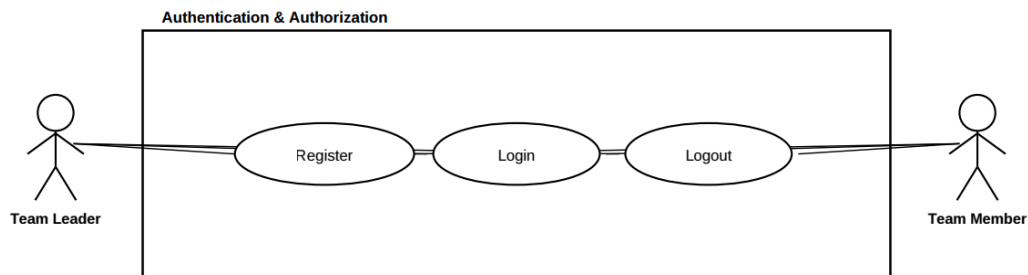
Role	Capabilities	Restrictions
Team Leader	Create workspace, invite members, create/edit/delete tasks, verify task completion, view all logs and contributions	Cannot demote self without assigning a new leader
Team Member	View assigned tasks, update own task status, post comments, access shared resources	Cannot create tasks, verify tasks, or manage members

3. UML Diagrams

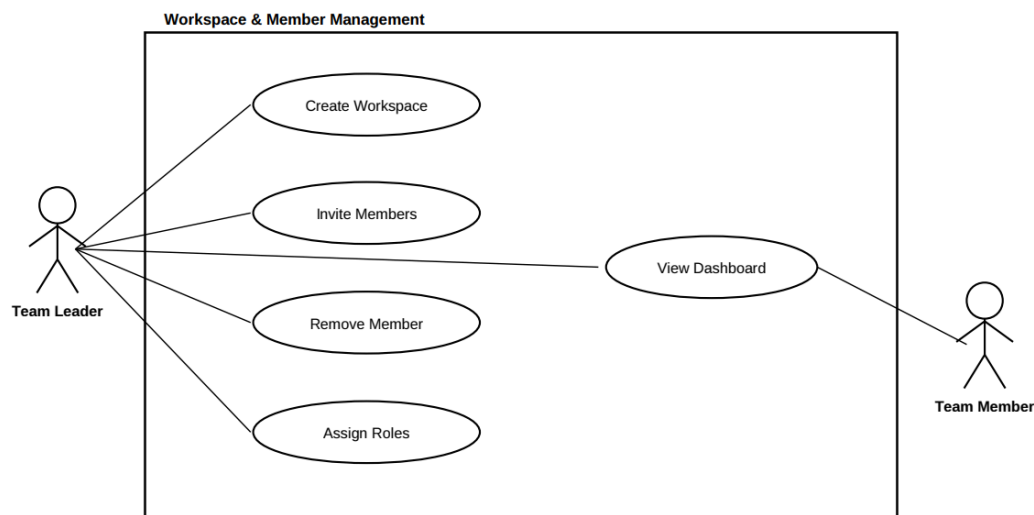
3.1 Use Case Diagram

Taskora has two actors: Team Leader and Team Member. Both actors share authentication and resource access. The Team Leader has elevated privileges for workspace administration and task verification.

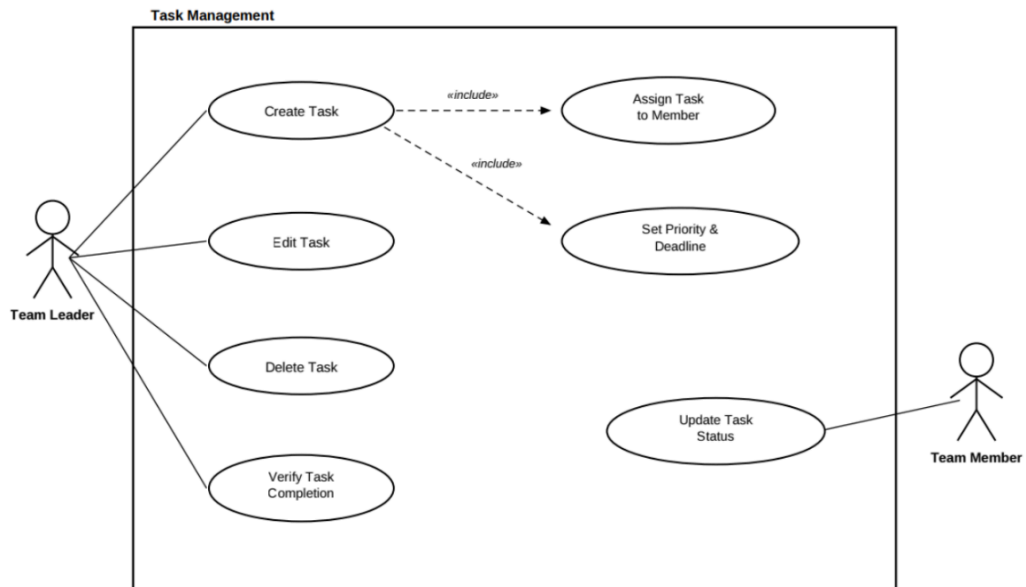
UC-01 AUTHENTICATION & AUTHORIZATION



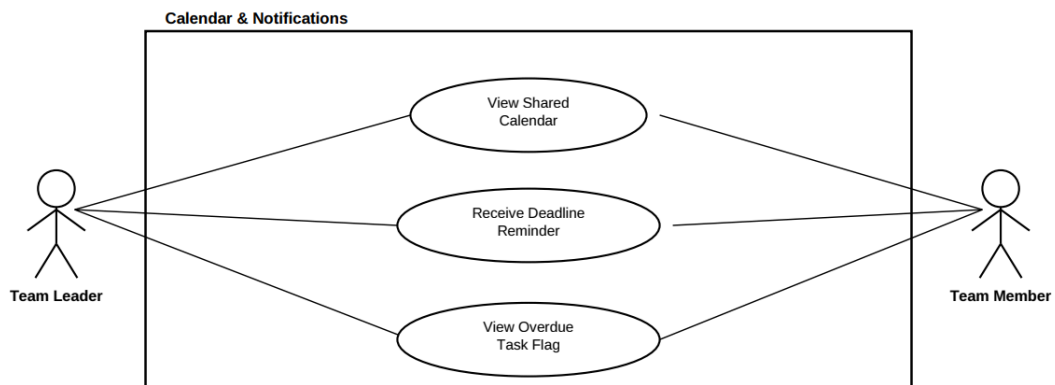
UC-02 WORKSPACE & MEMBER MANAGEMENT



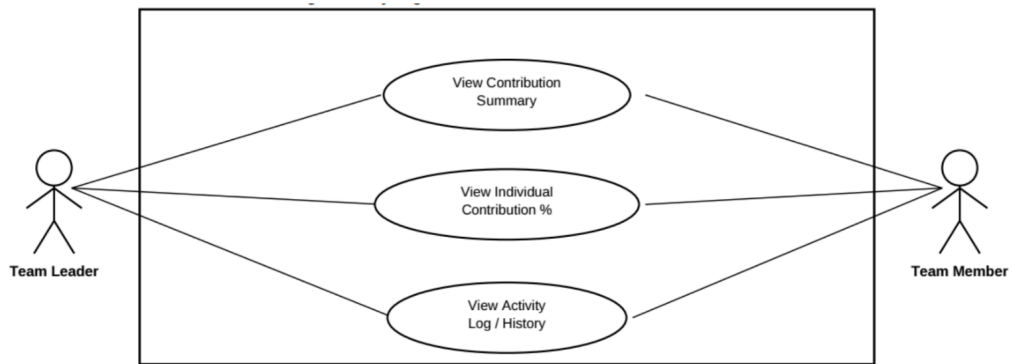
UC-03 TASK MANAGEMENT



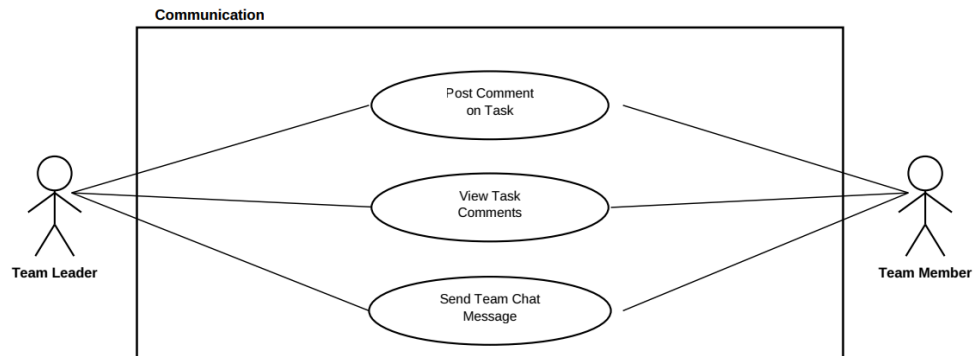
UC-04 CALENDAR & NOTIFICATIONS



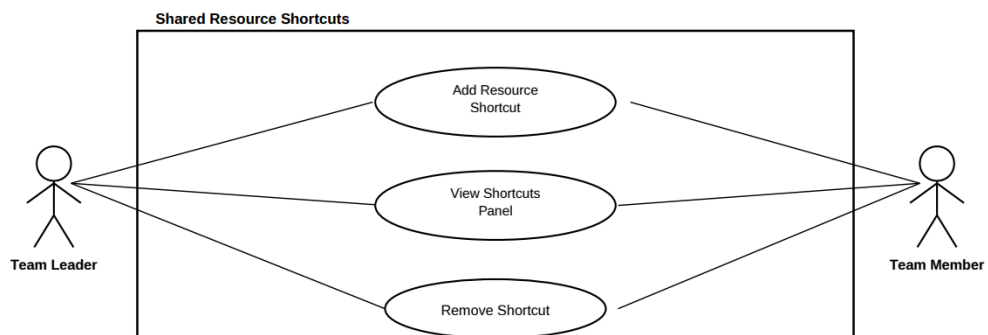
UC-05 CONTRIBUTION TRACKING & ACTIVITY LOG



UC-06 COMMUNICATION

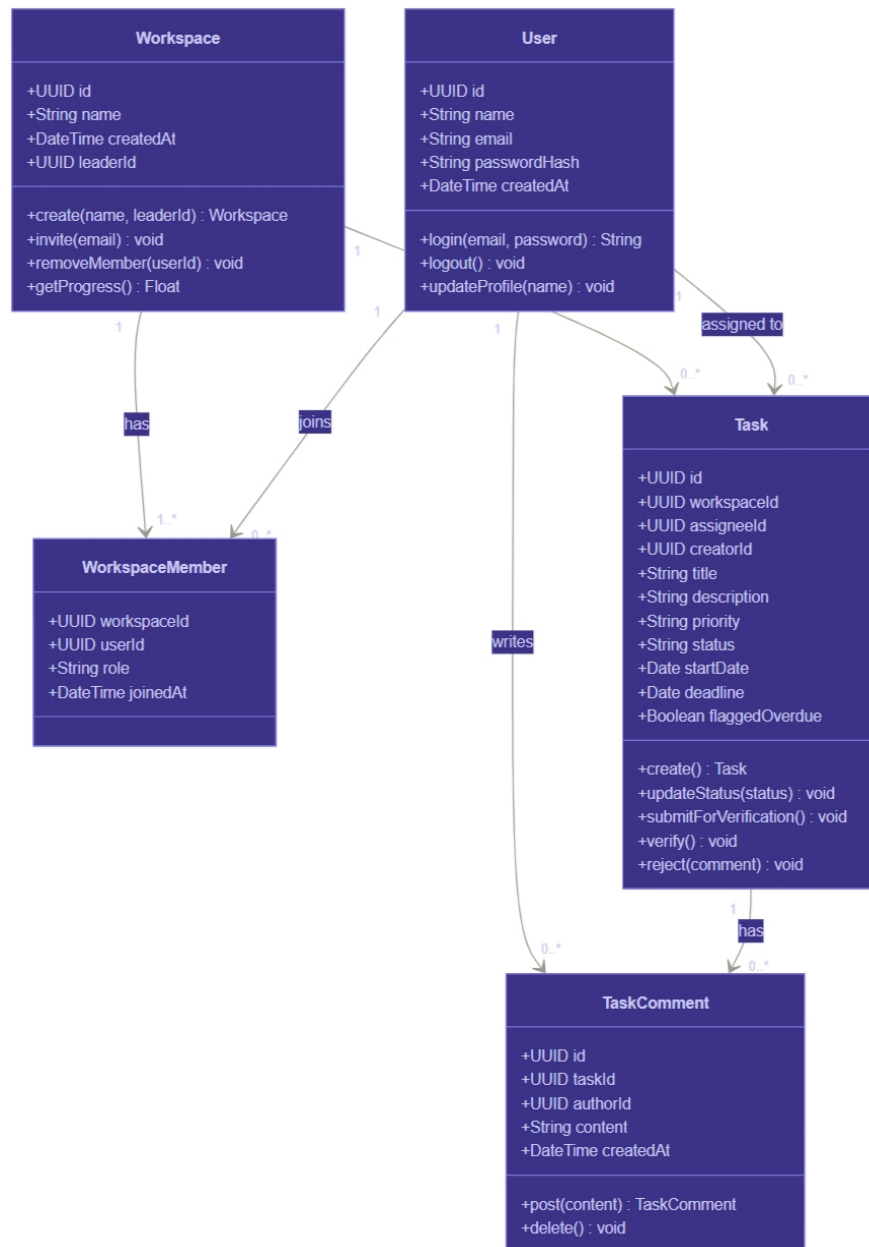


UC-07 SHARED RESOURCE SHORTCUTS

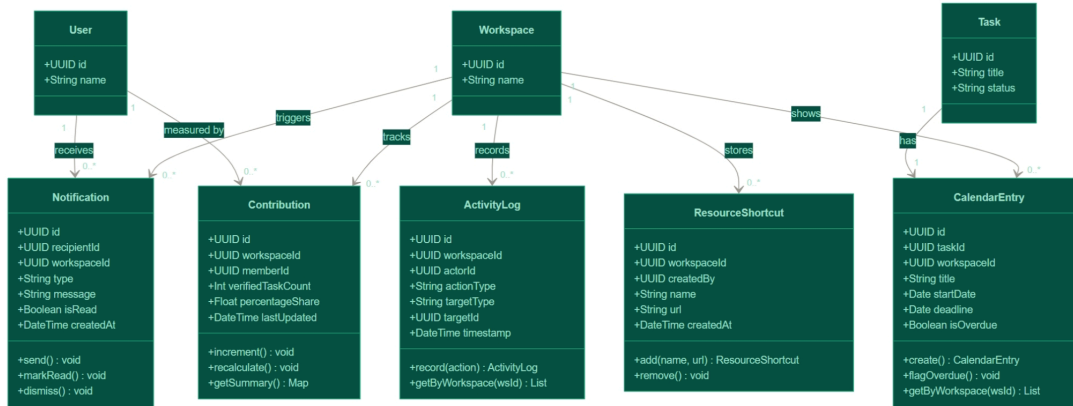


3.2 Class Diagram

3.2.1 Core Domain

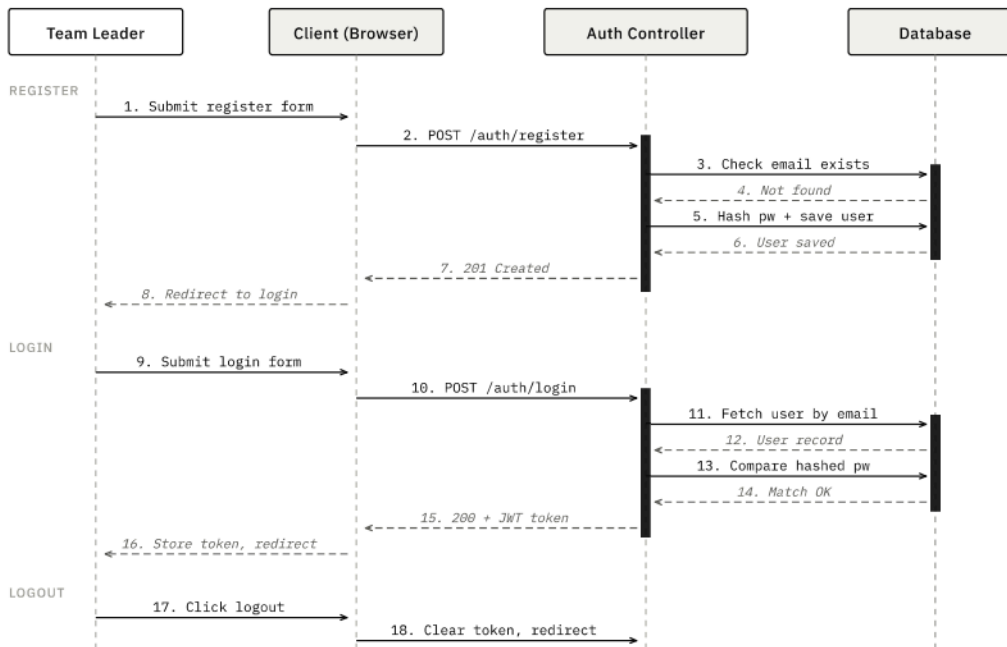


3.2.2 Supporting Domain

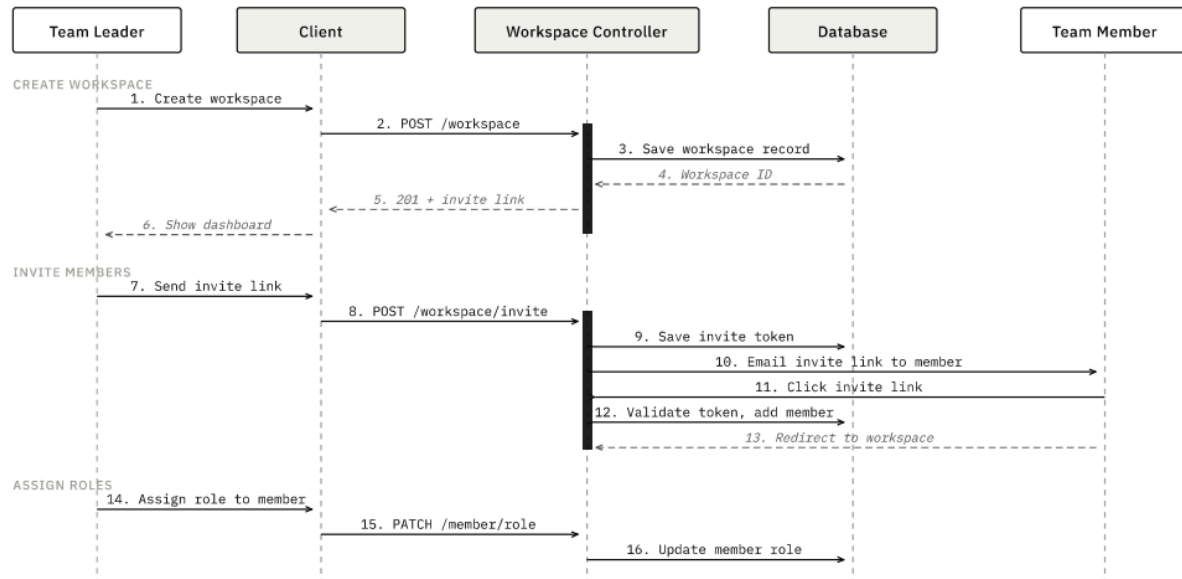


3.3 Sequence Diagrams

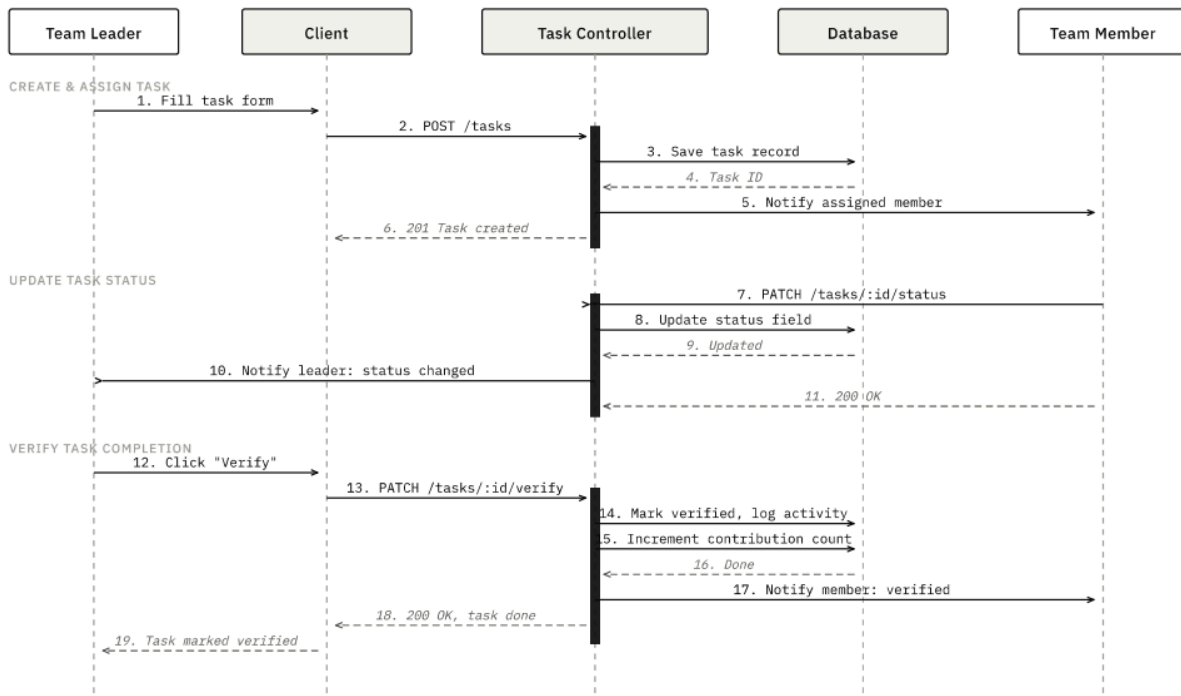
SD-01 AUTHENTICATION & AUTHORIZATION



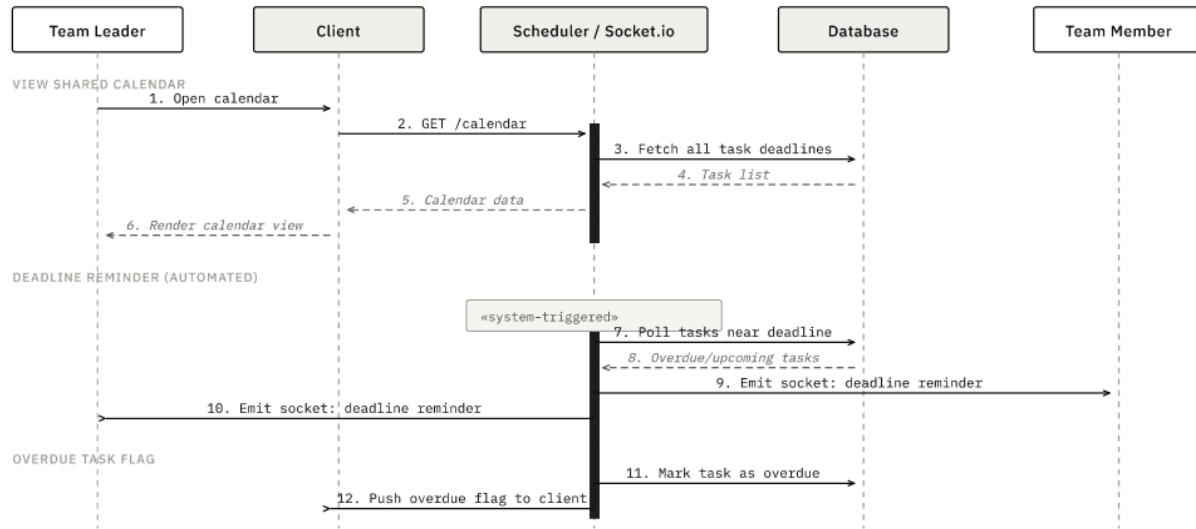
SD-02 WORKSPACE & MEMBER MANAGEMENT



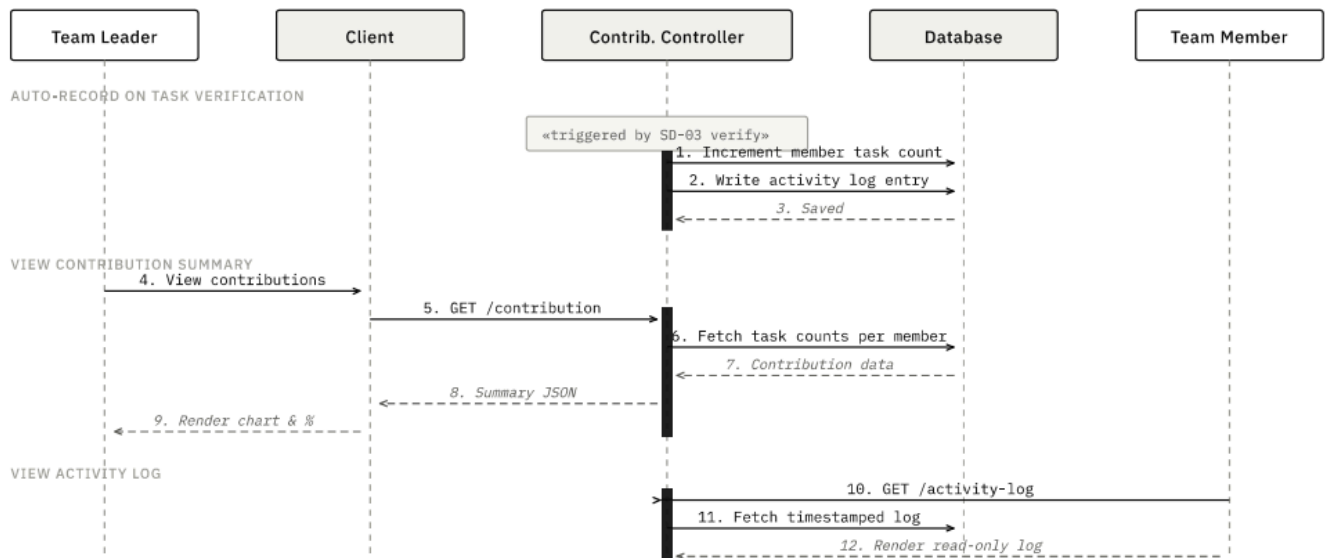
SD-03 TASK MANAGEMENT



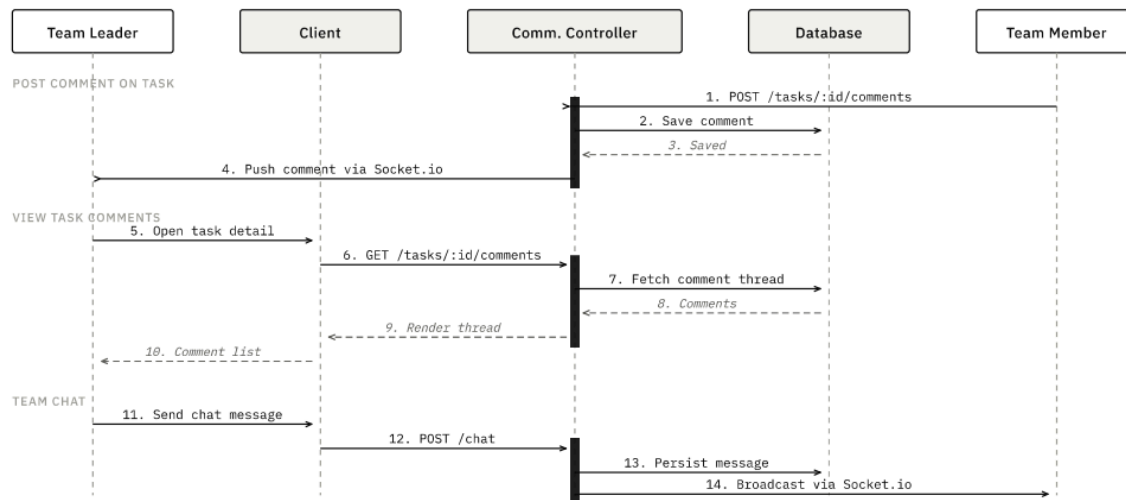
SD-04 CALENDAR & NOTIFICATIONS



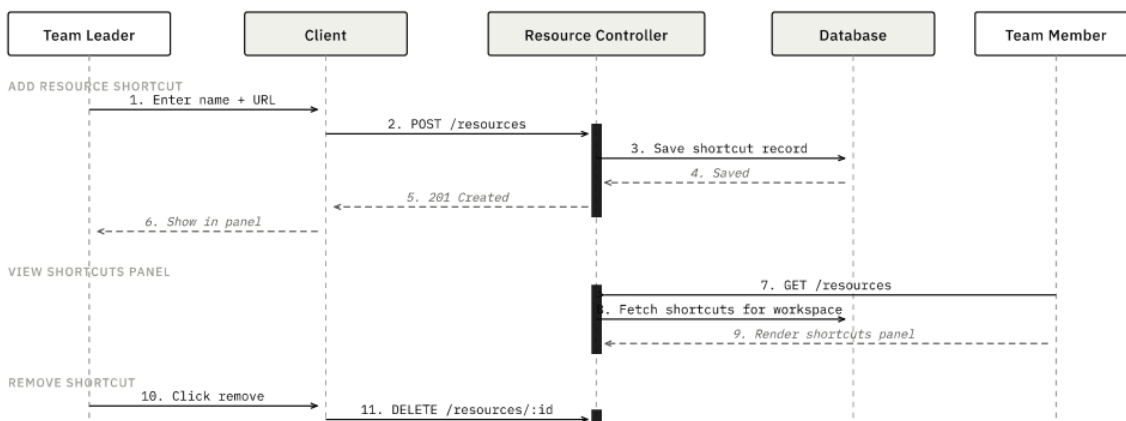
SD-05 CONTRIBUTION TRACKING & ACTIVITY LOG



SD-06 COMMUNICATION

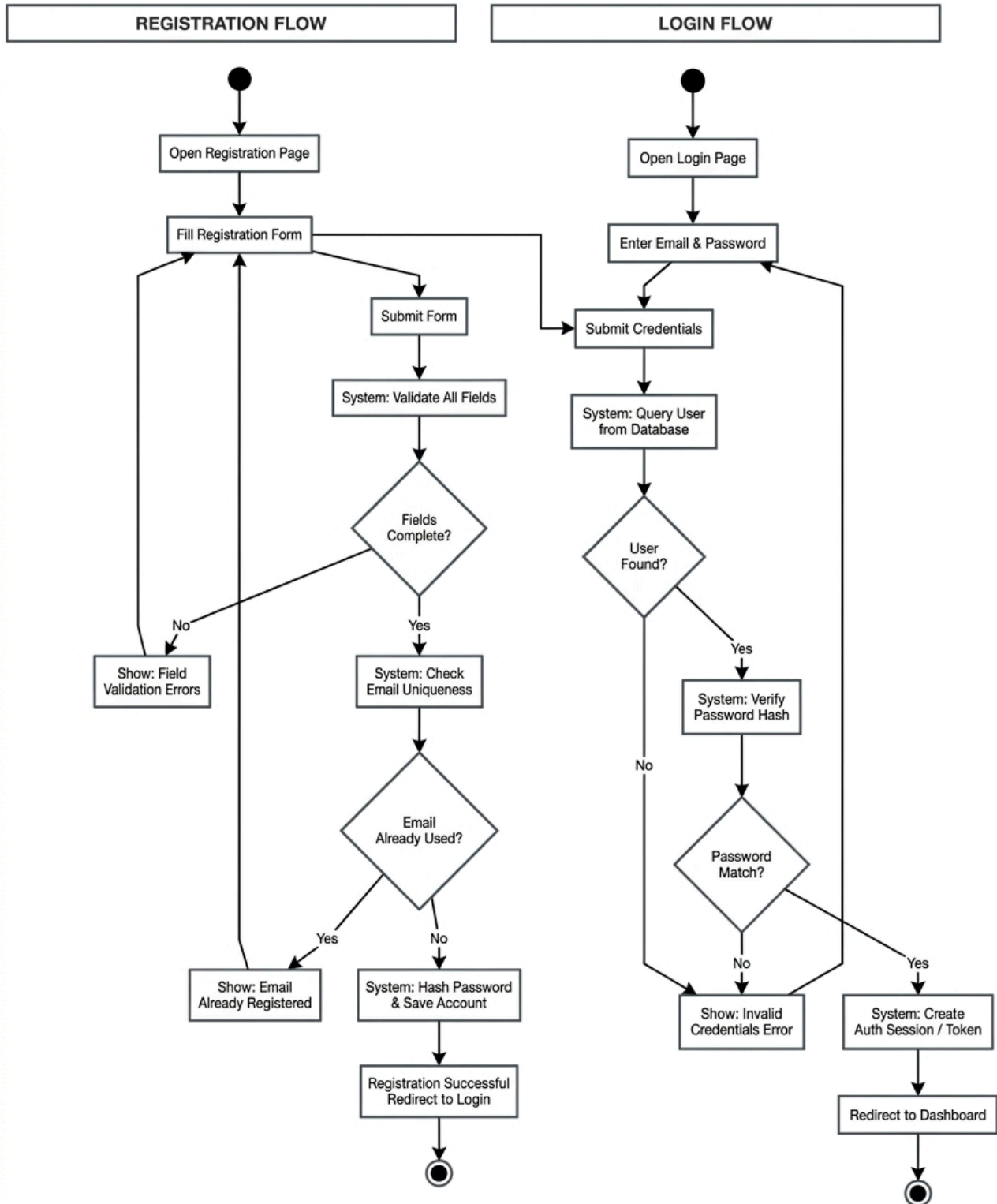


SD-07 SHARED RESOURCE SHORTCUTS

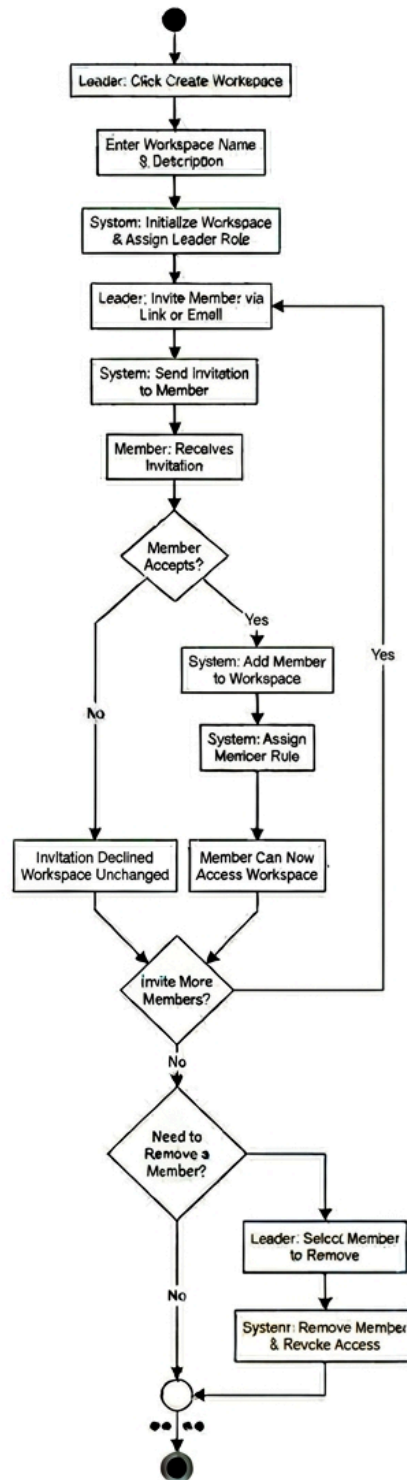


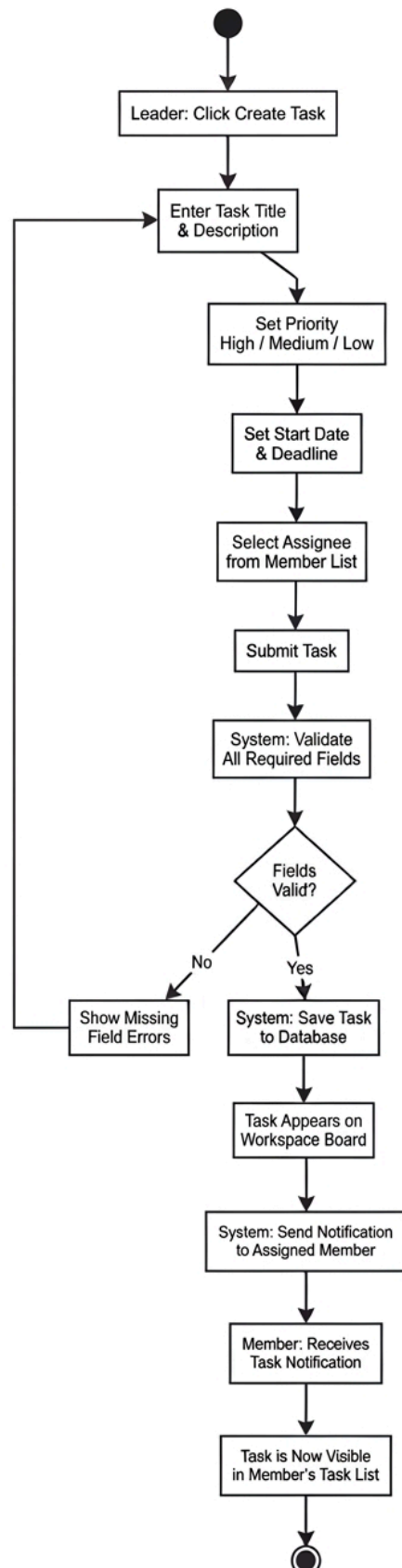
3.4 Activity Diagram

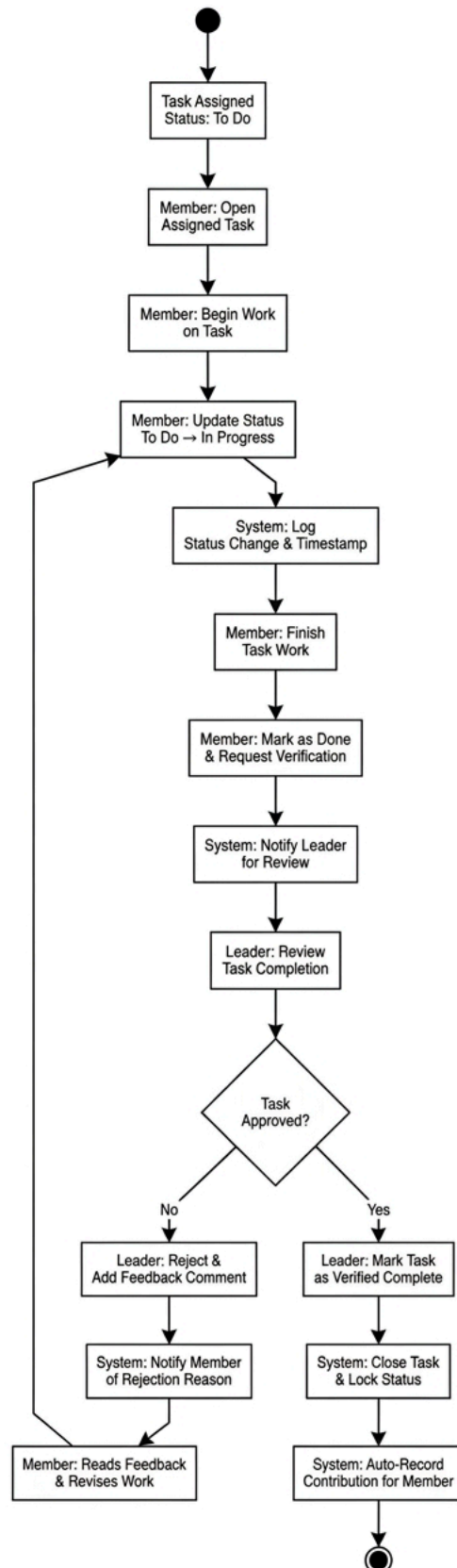
AD-01 **USER AUTHENTICATION**

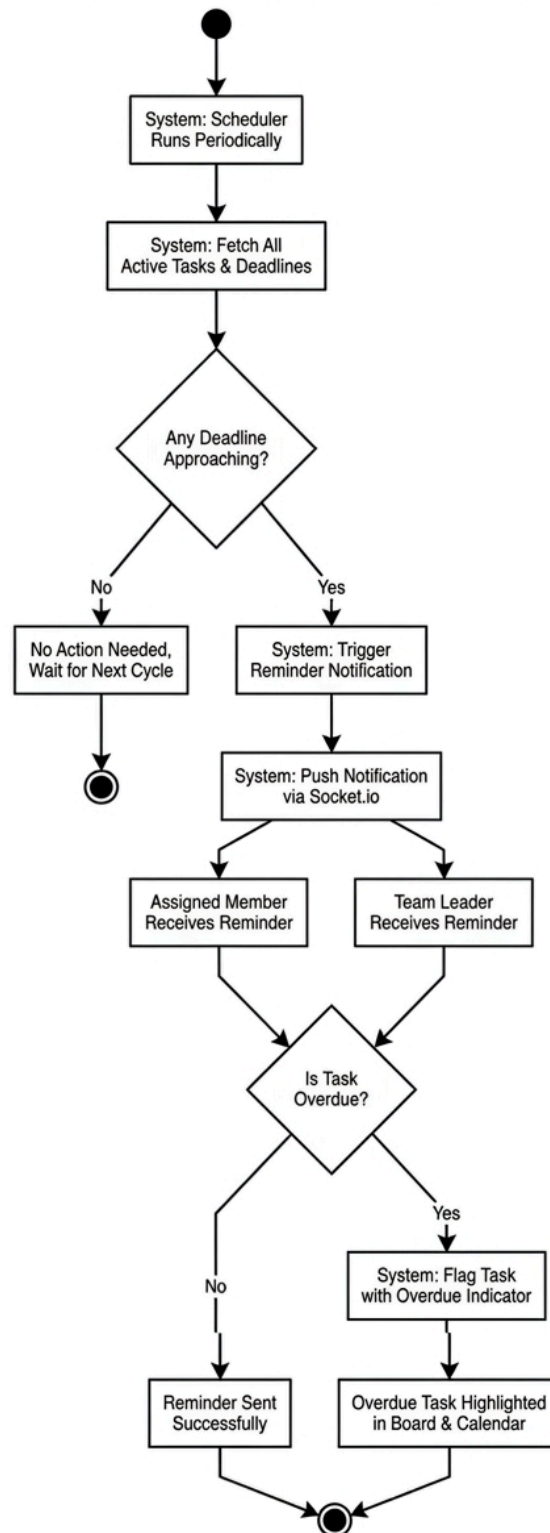


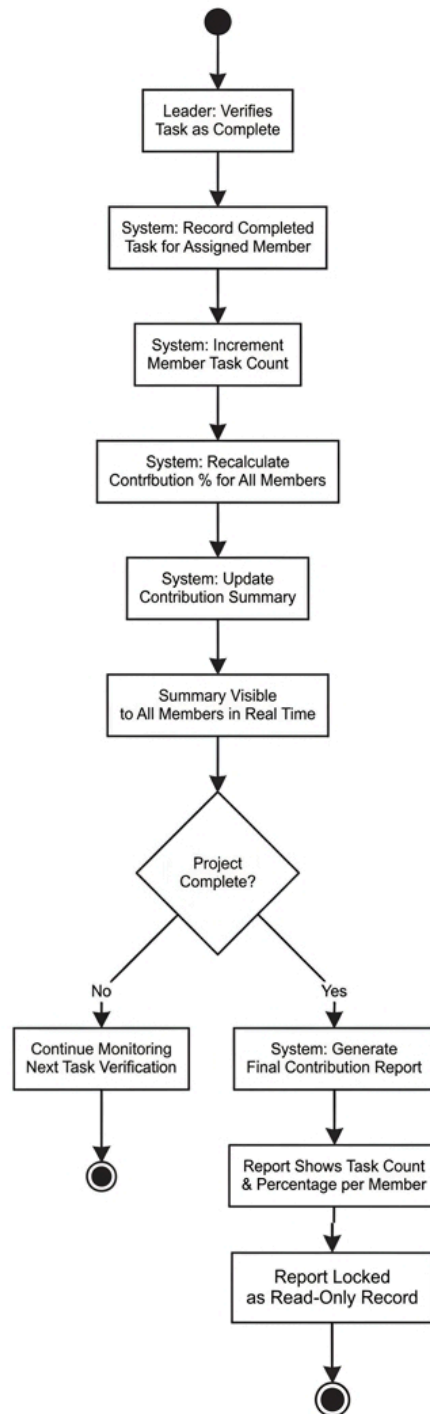
AD-02 WORKSPACE & MEMBER MANAGEMENT



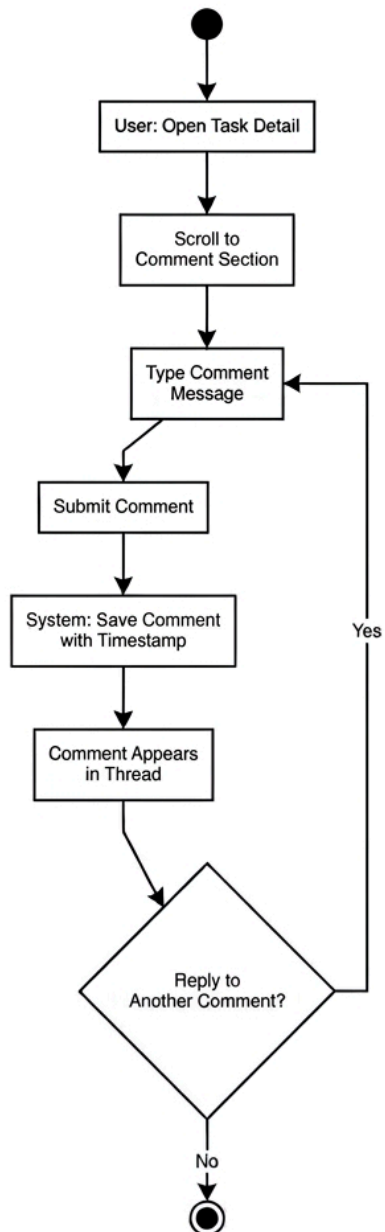




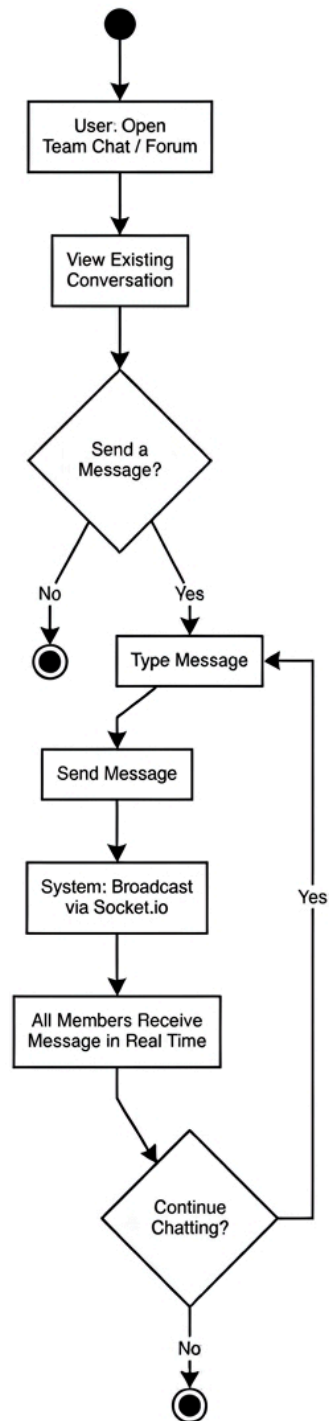


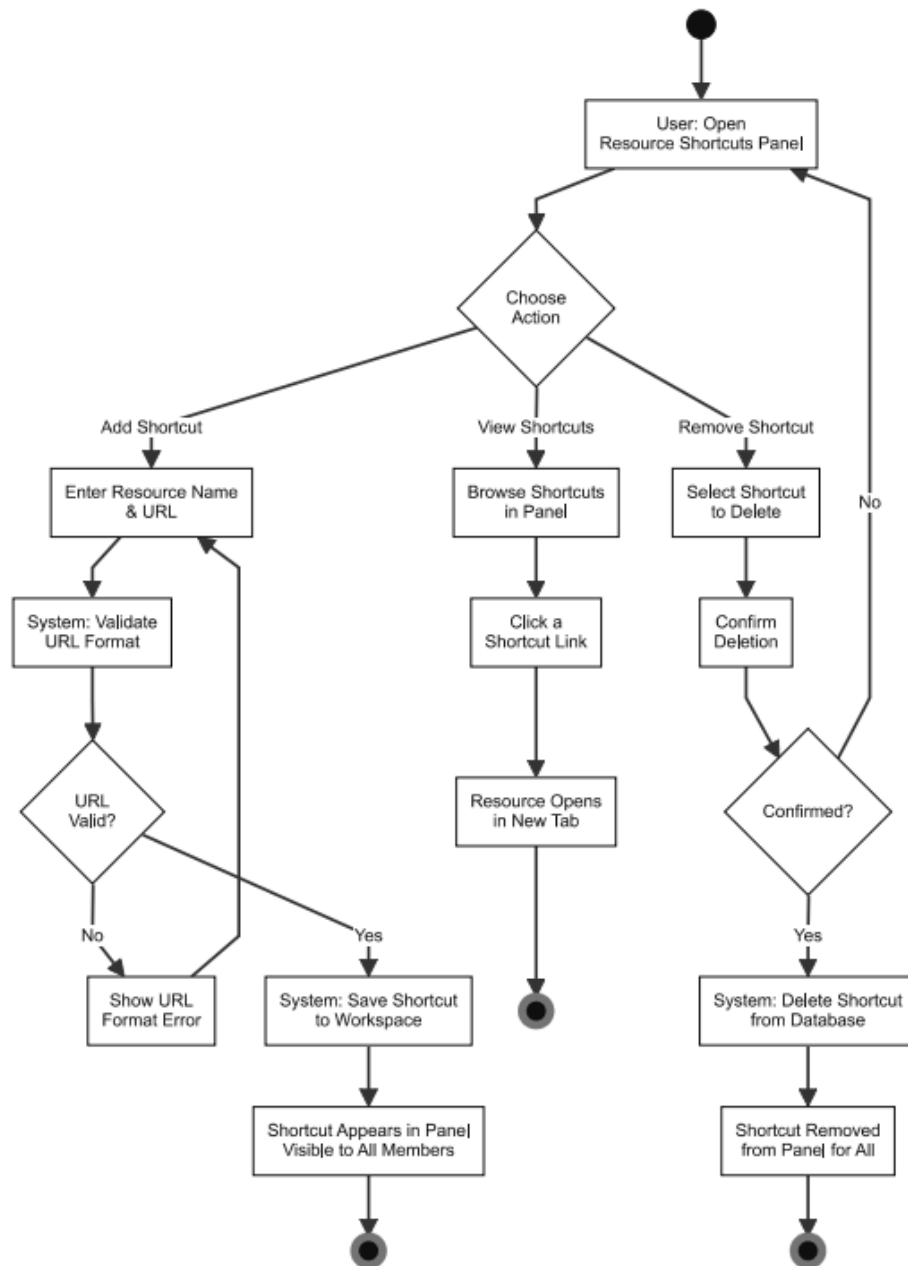


TASK COMMENT FLOW

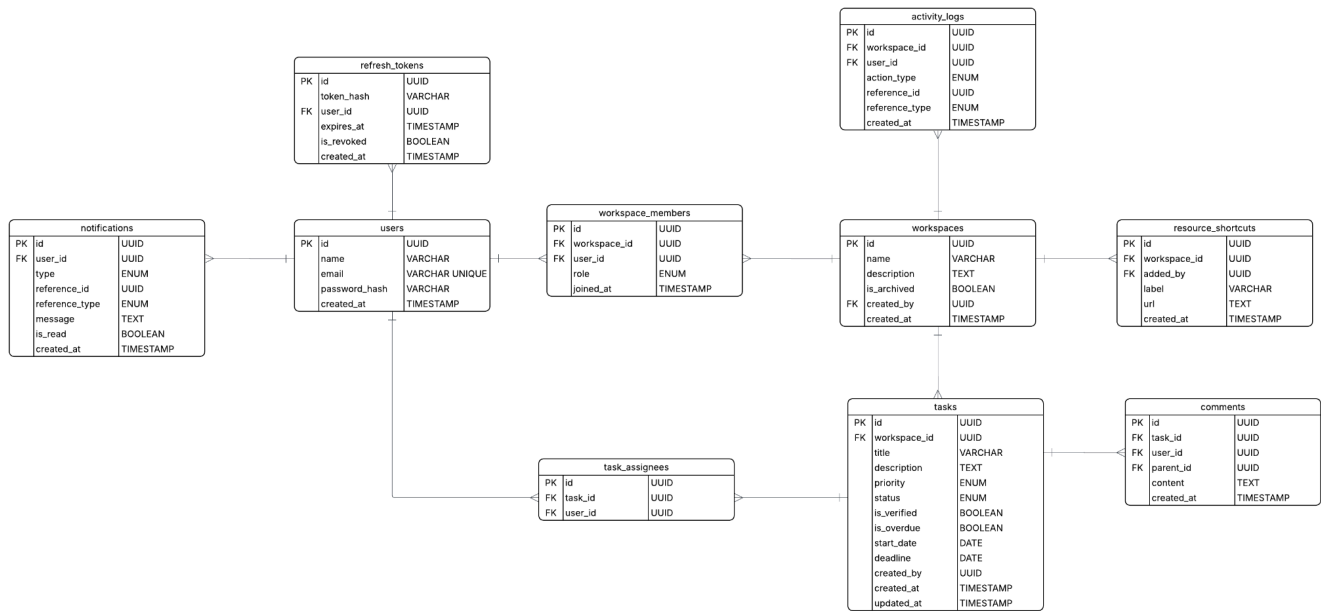


TEAM CHAT FLOW





4. Database Schema



5. API Design

5.1 Authentication

Method	Endpoint	Access	Description
POST	/auth/register	Public	Create new user account
POST	/auth/login	Public	Login and receive JWT
POST	/auth/logout	Authenticated	Invalidate session

5.2 Workspaces

Method	Endpoint	Access	Description
POST	/workspaces	Authenticated	Create new workspace
GET	/workspaces/:id	Member	Get workspace details + dashboard data
POST	/workspaces/:id/invite	Leader	Invite member by email or generate link
DELETE	/workspaces/:id/members/:uid	Leader	Remove member from workspace

5.3 Tasks

Method	Endpoint	Access	Description
GET	/workspaces/:id/tasks	Member	List all tasks in workspace
POST	/workspaces/:id/tasks	Leader	Create new task
PATCH	/tasks/:id	Leader	Edit task details
DELETE	/tasks/:id	Leader	Delete task
PATCH	/tasks/:id/status	Assignee	Update status (to <u>do</u> → <u>in progress</u>)
PATCH	/tasks/:id/submit	Assignee	Submit task for verification
PATCH	/tasks/:id/verify	Leader	Verify or reject submitted task

5.4 Other Modules

Method	Endpoint	Access	Description
GET	/workspaces/:id/calendar	Member	Get calendar events for workspace
GET	/workspaces/:id/contributions	Member	Get contribution summary
GET	/workspaces/:id/logs	Member	Paginated activity log
POST	/tasks/:id/comments	Member	Post comment on task
DELETE	/comments/:id	Author / Leader	Delete comment
GET	/notifications	Authenticated	Get user notifications
PATCH	/notifications/:id/read	Authenticated	Mark notification as read
POST	/workspaces/:id/resources	Member	Add resource shortcut
DELETE	/resources/:id	Creator / Leader	Remove resource shortcut